

Documentação do Projeto — CostaDH (Site + Backoffice)

Cliente: Costa Desenvolvimento Humano (CostaDH) — <https://costadh.com.br/> **Tipo de documento:** Passagem de conhecimento (handover técnico e funcional) **Data de elaboração:** 04/09/2026 **Destinatário:** Desenvolvedor(a) que assumirá a manutenção e evolução do projeto

Sumário

1. [Visão geral do projeto](#)
2. [Parte 1 — Funcionalidades e regras de negócio](#) - 2.1 [Site público](#) - 2.2 [Backoffice \(área administrativa\)](#) - 2.3 [Usuários e permissões](#) - 2.4 [Fluxo de e-mails e Shortlist](#)
3. [Parte 2 — Pontos técnicos do projeto](#) - 3.1 [Stack e versões](#) - 3.2 [Estrutura de pastas relevante](#) - 3.3 [Rotas principais](#) - 3.4 [Banco de dados](#) - 3.5 [Armazenamento de arquivos \(currículos e uploads\)](#) - 3.6 [Configurações sensíveis \(.env\)](#) - 3.7 [Passo a passo de restauração em ambiente local](#)
4. [Demanda pendente: repaginação do site](#)
5. [Observações finais e recomendações](#)

1. Visão geral do projeto

O projeto é composto por **duas frentes que compartilham o mesmo código-base Laravel**:

1. **Site público (institucional + captação de talentos)** — responsivo, com página única (home) contendo todo o conteúdo institucional da consultoria, mais telas de candidatura a vagas, cadastro no banco de currículos, visualização pública de shortlists e política de privacidade.
2. **Backoffice (painel administrativo)** — utilizado pela equipe da CostaDH para gerenciar vagas, avaliar candidatos, disparar e-mails de shortlist/materiais para empresas clientes e administrar usuários do sistema.

O conteúdo institucional da home **não é gerenciado via backoffice** — qualquer alteração de texto, imagem ou seção da home deve ser feita diretamente no código (arquivos Blade). Já as **vagas** são 100% dinâmicas, cadastradas e geridas pelo backoffice.

Parte 1 — Funcionalidades e regras de negócio

2.1 Site público

a) Home (página principal)

- **Rota:** /
- **Controller:** App\Http\Controllers\SiteController@index_site
- **View:** resources/views/site/index.blade.php

A home é uma página única (single page) dividida nas seções abaixo. Para alterar qualquer texto, imagem ou bloco visual da home, o desenvolvedor deve editar diretamente essa view, localizando a seção pelo `id` /comentário HTML correspondente:

Seção	Âncora / <code>id</code> no HTML	Conteúdo	Observação
Cabeçalho (Header)	<code>#header</code>	Logo + menu hambúrguer	Compartilhado visualmente com o restante do site
Menu de navegação	<code>#menu</code>	Links: Home, Sobre, Soluções, Oportunidades, Contato, Login	Menu do tipo overlay/lateral
Banner principal	<code>#banner</code>	Hero com título "COSTA DESENVOLVIMENTO HUMANO"	—
Seção Sobre	<code>#sobre_a_costa_dh</code>	Foto circular + texto de apresentação de Juliana Costa (2 colunas)	—
Seção Soluções (bloco 1)	<code>#solucoes</code>	3 cards: Cultura e Diversidade, Treinamento e Dinâmicas, Ações de Saúde Mental	—
Seção Soluções (bloco 2)	logo após <code>#solucoes</code>	3 cards: Metodologias, Performance, Seleção de Pessoas	Mesmo padrão visual do bloco 1
Seção Oportunidades	<code>#oportunidades</code>	Tabela de vagas ativas (Vaga, Área, Nível, Localização, Visualizar) + CTA para o banco de talentos	Conteúdo dinâmico — vem do banco (<code>vagas</code> com <code>status = 'habilitada'</code>)
Seção O Manifesto	logo após <code>#oportunidades</code> (sem <code>id</code> próprio)	Bloco textual institucional, sem imagem	—
Seção Depoimentos	logo em seguida (sem <code>id</code> próprio)	3 cards de depoimentos de clientes (nome + cargo)	—
Seção Clientes e Parceiros	<code>.clientes-parceiros</code>	Grid com 18 logos (<code>new_brands/01.png</code> até <code>18.png</code>)	Ver pasta de imagens abaixo

Seção	Âncora / <code>id</code> no HTML	Conteúdo	Observação
Banner CTA (Contato)	<code>#banner2</code>	"Qual o seu desafio?" com botão para WhatsApp	Número de WhatsApp vem de <code>config('services.whatsapp_contact')</code> , configurável via <code>.env</code> (<code>WHATSAPP_CONTACT</code>)
Rodapé	<code>#footer</code>	Arte decorativa lateral + logo + ícones sociais (Instagram, LinkedIn) + copyright	—

Imagens da home: ficam em `public/site/images/`, com destaque para: - `public/site/images/new_brands/` → os 18 logos de clientes/parceiros exibidos na seção "Clientes e Parceiros" (arquivos `01.png` a `18.png`). - Demais arquivos soltos da pasta (`banner*.png`, `pic0*.jpg`, `ju01.jpeg`, `sol.jpeg`, `avatar*.png/jpg`, etc.) são usados nas seções de banner, sobre, soluções e depoimentos.

Assets de estilo/script da home (template atual "Urban"/"Intensify" by TEMPLATED): - CSS:

`public/site/assets/css/` - JS: `public/site/assets/js/` - Fontes: `public/site/assets/fonts/`

Esses assets (CSS/JS) são o alvo da demanda de repaginação visual descrita na seção 4 deste documento.

b) Detalhe da vaga + formulário de candidatura

- **Rota:** `/vaga/{id}`
- **Controller:** `SiteController@form_cadastro_vaga`
- **View:** `resources/views/site/form_vaga.blade.php`

Exibe os detalhes completos de uma vaga (título, área, nível, localização, descrição) e um formulário de candidatura com: nome, e-mail, telefone, sexo, UF, cidade, upload de currículo e aceite de termos LGPD. Após o envio bem-sucedido, uma mensagem de agradecimento é exibida na mesma tela (condicional via `session('success')`).

Regra de negócio do envio (`SiteController@salvar_candidatura`, rota `POST /salvar_candidatura`): 1. Verifica se já existe um **candidato** cadastrado com aquele e-mail (tabela `candidato`). 2. Se existir: atualiza os dados do candidato (e o currículo, se um novo arquivo foi enviado) e, em seguida, verifica se ele já possui uma **candidatura** para aquela vaga (tabela `candidatura`); se não tiver, cria a candidatura. 3. Se não existir: cria um novo registro em `candidato` e, em seguida, cria a `candidatura` vinculando `candidato_id` e `vaga_id`. 4. Se o campo oculto `vaga_id` vier como `'false'`, entende-se que é apenas um cadastro de currículo avulso (sem vaga), e nenhuma candidatura é criada — esse é o mesmo endpoint usado pelo formulário de banco de currículos (item c). 5. Currículos enviados são salvos fisicamente em `public/uploads/curriculos/`, com o nome do arquivo baseado em timestamp (`time()` . `'.'` . extensão).

c) Banco de talentos (cadastro de currículo avulso)

- **Rota:** `/cadastrar_curriculo`

- **Controller:** `SiteController@form_cadastro`
- **View:** `resources/views/site/form_curriculo.blade.php`

Formulário com os mesmos campos da candidatura a vaga, mais um campo de área de interesse. Utiliza o **mesmo endpoint** `salvar_candidatura`, enviando `vaga_id = 'false'` para indicar que não há vínculo com uma vaga específica.

d) Visualização de Shortlist (link exclusivo)

- **Rota:** `/shortlist/link-exclusivo/{identificador}`
- **Controller:** `HomeController@visualizarPublico`
- **View:** `resources/views/site/view_documentos_exclusivo.blade.php`

Tela acessada por **empresas/clientes externos** que recebem o link por e-mail (ver item 2.4). Exibe a lista de currículos/documentos vinculados àquele envio, com botões de download individuais. O acesso é feito por um identificador público (`identificador_publico`, gerado via `base64_encode(uniqid())`), sem necessidade de login — por isso é uma tela de **alta prioridade de imagem/marca**, pois é vista por terceiros fora do backoffice.

Downloads: - Planilha do envio: `GET /download-exclusivo/{arquivo}` → `HomeController@baixarArquivo` - Currículo/documento individual: `GET /download-curriculos-exclusivo/{arquivo}` → `HomeController@baixarCurriculo`

Ambos fazem download a partir do disco `public` do Laravel (`storage/app/public/documentos/...`).

e) Política de Privacidade

- **Rota:** `/privacidade-de-dados-detahes`
- **Controller:** `SiteController@privacidade`
- **View:** `resources/views/site/detalhe_privacidade.blade.php`

Página estática com o texto completo de privacidade/LGPD.

2.2 Backoffice (área administrativa)

Acesso via login (Laravel Auth padrão, rotas geradas por `Auth::routes(['register' => false])` — **não há autoregistro público**, usuários são criados apenas pelo próprio backoffice). O menu/layout administrativo usa o pacote **AdminLTE** (via `jeroennoten/laravel-adminlte`).

a) Gestão de vagas

- Cadastro: `GET /admin/cad-vagas` → view `resources/views/vagas/cad_vagas.blade.php`
- Listagem: `GET /admin/list-vagas` → `resources/views/vagas/list_vagas.blade.php`
- Salvar (criar/editar): `POST /admin/salvar-vaga` → `VagaController@salvar_vaga`
- Visualizar/editar: `GET /admin/visualizar-vaga/{id}` → `resources/views/vagas/edit_vagas.blade.php`

Regras de negócio: - Ao salvar uma vaga nova, o sistema gera um `slug` no formato `{id}-{slug-do-titulo}` (usado como identificador amigável, embora as rotas de vaga usem o `id` puro em `/vaga/{id}`). - Se o campo `status` não vier marcado no formulário, a vaga é salva como `desabilitada`. - Apenas vagas com `status = 'habilitada'` aparecem na seção "Oportunidades" da home e no formulário de candidatura.

b) Painel de candidaturas (inscrições em vagas)

- **Rota principal:** `GET/POST /admin/index-painel` → `CandidatoController@index_painel`
- **Rota de filtro:** `GET/POST /admin/index-painel-filtro` → `CandidatoController@index_painel_filtro`
- **View:** `resources/views/painel/index.blade.php`

Lista todas as candidaturas (join entre `candidatura`, `candidato` e `vagas`), com paginação (50 por página) e **filtros combináveis**: - Nome do candidato (quando preenchido, os demais filtros são ignorados — busca isolada por nome). - Título da vaga, UF da vaga, cidade da vaga, área, nível, escolaridade, tipo de contrato. - UF e cidade do candidato.

Há também um endpoint auxiliar `GET /admin/contar-candidatos-vaga` (`CandidatoController@contar_candidatos_vaga`) que retorna via JSON o total de candidaturas de uma vaga específica — usado tipicamente em contadores/badges na tela de vagas.

c) Avaliação do candidato (detalhe + timeline/evolução)

- **Rota:** `GET/POST /admin/detalhe-candidato/{id}` → `CandidatoController@detalhe_candidato`
- **View:** `resources/views/painel/detalhe.blade.php`

Mostra o histórico de **todas as candidaturas** daquele candidato (pode ter se candidatado a mais de uma vaga), cada uma com seu `status` e `observacoes`, ordenadas da mais recente para a mais antiga — é essa listagem que alimenta a **timeline visual de evolução** exibida na tela.

Duas ações de atualização a partir dessa tela: - `POST /admin/evoluir-candidato` → `CandidatoController@evoluir_candidato_vaga`: atualiza o `status` (etapa do processo seletivo) e observações de uma **candidatura específica** (`candidatura.id`). - `POST /admin/dados-candidato` → `CandidatoController@dados_candidato_vaga`: atualiza os **dados cadastrais** do candidato (tabela `candidato`), independentemente da vaga.

O conjunto de status/etapas possíveis (ex.: triagem, entrevista, aprovado, reprovado etc.) é tratado como texto livre no campo `candidatura.status` — não há uma tabela/enum fixo de etapas no banco atual. Isso é importante para quem for evoluir essa funcionalidade futuramente.

d) Banco de currículos (candidatos sem foco em vaga específica)

- **Rota:** `GET/POST /admin/index-painel-cand` → `CandidatoController@index_painel_cand`

- **Rota de filtro:** GET/POST /admin/index-painel-cand-filtro → CandidatoController@index_painel_cand_filtro
- **View:** resources/views/painel/curriculos.blade.php

Lista **todos** os candidatos cadastrados na tabela `candidato` (com ou sem candidatura vinculada), com filtros por nome, período de cadastro (`data_inicio` / `data_fim`), UF e cidade. Serve para a equipe de RH pesquisar no banco de talentos avulso.

e) Disparo de Shortlist / Materiais para clientes (empresas)

Essa é a funcionalidade de comunicação com as **empresas contratantes** dos processos seletivos da CostaDH.

- **Shortlist (resultado do processo seletivo):**
 - Tela: GET /admin/envio-resultados → resources/views/painel/envio_resultados.blade.php
 - Disparo: POST /admin/disparar-documentos → HomeController@disparar_documentos
 - E-mail: App\Mail\EnvioDocumentosMail
- **Materiais diversos (outro tipo de comunicação, mesmo mecanismo):**
 - Tela: GET /admin/envio-materiais → resources/views/painel/envio_materiais.blade.php
 - Disparo: POST /admin/disparar-materiais → HomeController@disparar_materiais
 - E-mail: App\Mail\EnvioMateriaisMail

Fluxo comum aos dois: 1. O operador informa nome do cliente, um ou mais e-mails (separados por vírgula), mensagem personalizada, uma planilha opcional (`.xlsx` / `.xls`) e uma lista de currículos/documentos (cada um com nome, arquivo e tipo). 2. O sistema gera um `identificador_publico` único (`base64_encode(uniqid())`) e grava um registro em `documento_envios` (`tipo_envio` = `resultado` ou `material_diverso`). 3. Cada currículo/documento anexado é salvo em `documento_curriculos`, vinculado ao envio. 4. Um **link público** é gerado (`route('documentos.publico', $identificador)`), apontando para a tela do item 2.1-d. 5. O e-mail é disparado para **cada endereço informado** (separando por vírgula), contendo o link e a mensagem customizada. 6. As telas de listagem de envios (GET /documentos/envios/listar e GET /materiais/envios/listar , ambas em HomeController) retornam em JSON os últimos 50 envios — usado para alimentar tabelas/históricos na interface do backoffice.

f) Gestão de usuários do backoffice

- Cadastro: GET /admin/cad-users → resources/views/users/cad_users.blade.php
- Listagem: GET /admin/list-users → resources/views/users/list_users.blade.php
- Salvar (criar/editar): POST /admin/salvar-user → UserController@salvar_user
- Visualizar/editar: GET /admin/visualizar-user/{id} → resources/views/users/edit_users.blade.php
- Excluir: GET /admin/deletar-user/{id} → UserController@deletar_user

2.3 Usuários e permissões

A tabela `users` possui, além dos campos padrão do Laravel, as colunas `status` (`Habilitado` / `Desabilitado` , controla se o usuário consegue logar/operar) e `super_admin` (booleano, `0` ou `1`).

Regras de negócio (`UserController@salvar_user`): - **Qualquer usuário logado** pode editar seu próprio nome, e-mail e senha. - **Alterar a senha de outro usuário** só é permitido se quem estiver logado for `super_admin` . - **Marcar/desmarcar o campo `super_admin`** de um usuário só pode ser feito por quem já é `super_admin` . Usuários comuns não conseguem se autopromover nem promover terceiros. - Um usuário recém-criado sem o checkbox de status marcado nasce como `Desabilitado` (bloqueado até habilitação manual).

A coluna `super_admin` **não existe na migration original** de `users` — ela foi adicionada posteriormente via script SQL avulso: `alter_users_super_admin.sql`. O dump de banco entregue (`database/costad77_sistema.sql`) já contém essa coluna, então não é necessário rodar esse script novamente ao restaurar o projeto a partir do dump — ele só é relevante como histórico/documentação da evolução do schema.

2.4 Fluxo de e-mails e Shortlist

Resumo do fluxo ponta a ponta, do ponto de vista de negócio:

1. A CostaDH conduz um processo seletivo para uma empresa cliente.
2. Ao concluir a etapa (ou parte dela), o operador do backoffice acessa a tela de **Envio de Resultados** (Shortlist) e monta um pacote com os currículos/documentos dos candidatos selecionados, mais uma mensagem personalizada.
3. O sistema dispara um e-mail para a empresa cliente contendo um **link exclusivo** e público (não requer login).
4. A empresa cliente acessa o link e visualiza/baixa os currículos e documentos diretamente pelo navegador — essa tela pública é a "vitrine" da CostaDH para terceiros, por isso tem alta prioridade visual no projeto de repaginação (ver seção 4).
5. O mesmo mecanismo é reaproveitado para o envio de "materiais diversos", trocando apenas o template de e-mail e o rótulo do tipo de envio.

Parte 2 — Pontos técnicos do projeto

3.1 Stack e versões

Item	Detalhe
Framework	Laravel 8.75 (<code>laravel/framework: ^8.75</code>)

Item	Detalhe
PHP	^7.3 \ ^8.0 (compatível com PHP 7.3 a 8.x — o dump mostra uso recente com PHP 8.4, portanto o ambiente atual roda em PHP 8)
Banco de dados	MySQL (dump gerado via phpMyAdmin 5.2.3, MySQL Server 5.7.44)
Painel administrativo	AdminLTE 3 via jeroennoten/laravel-adminlte: ^3.9
Autenticação	laravel/ui (scaffolding clássico de login, sem registro público) + laravel/sanctum (instalado, mas sem uso de API tokens identificado nas rotas atuais)
CORS	fruitcake/laravel-cors
Build de assets do Laravel (admin)	Laravel Mix (webpack.mix.js , package.json)
Ambiente local usado até aqui	WAMP (c:\wamp64\www\dhcosta)

3.2 Estrutura de pastas relevante

```

app/
  Http/Controllers/
    SiteController.php      -> Home, candidatura, banco de currículos, privacidade
    HomeController.php      -> Painel inicial admin + Shortlist/Materiais + downloads público
    CandidatoController.php -> Painel de candidaturas, filtros, avaliação/timeline
    VagaController.php      -> CRUD de vagas
    UserController.php      -> CRUD de usuários do backoffice
  Models/
    Candidato.php, Candidatura.php, Vagas.php, User.php
    DocumentoEnvio.php, DocumentoCurriculo.php
  Mail/
    EnvioDocumentosMail.php, EnvioMateriaisMail.php
resources/views/
  site/      -> views públicas (home, form_vaga, form_curriculo, view_documentos_exclusivo, d
  painel/    -> views do backoffice (index = candidaturas, curriculos, detalhe, envio_resulta
  vagas/     -> CRUD de vagas (não listado em detalhe aqui, mas segue mesmo padrão do módulo
  users/     -> CRUD de usuários
  emails/    -> templates dos e-mails de Shortlist/Materiais
public/
  site/assets/{css,js,fonts}/ -> assets do template atual do site (alvo da repaginação)
  site/images/                -> imagens da home, incluindo new_brands/ (logos de clientes)
  uploads/curriculos/         -> currículos enviados via formulário de candidatura/banco de
  upload/curriculos/          -> pasta legada/alternativa (também presente no projeto)
  storage -> link simbólico para storage/app/public (ver seção 3.5)
database/
  costad77_sistema.sql        -> dump completo do banco de produção (enviado ao novo dev)
  migrations/                 -> apenas as migrations padrão do Laravel (ver observação abaixo
docs/
  escopo-repaginacao-site.doc/.pdf -> escopo funcional da repaginação visual

```

cronograma-repaginacao-site.doc/.pdf -> cronograma/sprints da repaginação (NÃO enviado ao documentacao-tecnica-projeto.md/.pdf -> este documento

Atenção — divergência entre migrations e banco real: a pasta `database/migrations/` contém apenas as migrations padrão do Laravel (`users` , `password_resets` , `failed_jobs` , `personal_access_tokens`). As tabelas de negócio (`candidato` , `candidatura` , `vagas` , `documento_envios` , `documento_curriculos`) não possuem migration correspondente no repositório — elas foram criadas diretamente no banco ao longo do tempo e só existem estruturadas no dump SQL. Ou seja, rodar `php artisan migrate` sozinho não recria o **schema completo**; é obrigatório importar o dump `costad77_sistema.sql` para ter o banco funcional (detalhes no passo a passo da seção 3.7).

3.3 Rotas principais

Arquivo: `routes/web.php`

Site público: | Método | Rota | Nome | Descrição | |---|---|---|---| | GET | `/` | `indexSite` | Home | | GET | `/vaga/{id?}` | `formCadastroVaga` | Detalhe da vaga + form de candidatura | | GET | `/cadastrar_curriculo` | `formCadastro` | Banco de currículos | | ANY | `/salvar_candidatura` | `salvarCandidatura` | Grava candidato/candidatura | | GET | `/privacidade-de-dados-detalhes` | `detPrivacidade` | Política de privacidade | | GET | `/shortlist/link-exclusivo/{identificador}` | `documentos.publico` | Shortlist pública | | GET | `/download-exclusivo/{arquivo}` | `documentos.download` | Download da planilha do envio | | GET | `/download-curriculos-exclusivo/{arquivo}` | `curriculos.download` | Download de currículo/documento do envio |

Backoffice (autenticado): | Método | Rota | Nome | Descrição | |---|---|---|---| | GET | `/admin/home` | `home` | Dashboard | | GET/POST | `/admin/cad-vagas` , `/admin/list-vagas` , `/admin/salvar-vaga` , `/admin/visualizar-vaga/{id?}` | `cadVagas` , `listVagas` , `salvarVaga` , `visualizarVaga` | CRUD de vagas | | GET/POST | `/admin/cad-users` , `/admin/list-users` , `/admin/salvar-user` , `/admin/visualizar-user/{id?}` , `/admin/deletar-user/{id?}` | `cadUsers` , `listUsers` , `salvarUser` , `visualizarUser` , `deletarUser` | CRUD de usuários | | ANY | `/admin/index-painel` , `/admin/index-painel-filtro` | `indexPainel` , `indexPainelFiltro` | Painel de candidaturas + filtro | | ANY | `/admin/detalhe-candidato/{id?}` | `detalheCandidato` | Detalhe/timeline do candidato | | POST | `/admin/evoluir-candidato` | `evoluirCandidatoVaga` | Atualiza status/etapa da candidatura | | POST | `/admin/dados-candidato` | `dadosCandidatoVaga` | Atualiza dados cadastrais do candidato | | ANY | `/admin/index-painel-cand` , `/admin/index-painel-cand-filtro` | `indexPainelCand` , `indexPainelCandFiltro` | Banco de currículos + filtro | | GET | `/admin/contar-candidatos-vaga` | `contarCandidatosVaga` | JSON com total de candidaturas de uma vaga | | GET/POST | `/admin/envio-resultados` , `/admin/disparar-documentos` | `envioResultados` , `dispararDocumentos` | Shortlist | | GET/POST | `/admin/envio-materiais` , `/admin/disparar-materiais` | `envioMateriais` , `dispararMateriais` | Materiais diversos | | GET | `/documentos/envios/listar` , `/materiais/envios/listar` | `documentos.envios.listar` , `materiais.envios.listar` | JSON de histórico de envios |

Utilitária: `GET /fix-storage-link` executa `Artisan::call('storage:link')` via navegador — útil em hospedagens sem acesso SSH/terminal, para recriar o link simbólico `public/storage`.

3.4 Banco de dados

Dump completo em [database/costad77_sistema.sql](#). Principais tabelas:

Tabela	Model	Finalidade
<code>candidato</code>	<code>Candidato</code>	Dados cadastrais de todas as pessoas que já se candidataram (a uma vaga ou ao banco de currículos). E-mail é usado como identificador de deduplicação.
<code>candidatura</code>	<code>Candidatura</code>	Relação N:N entre <code>candidato</code> e <code>vagas</code> , com <code>status</code> (etapa, texto livre) e <code>observacoes</code> .
<code>vagas</code>	<code>Vagas</code>	Vagas cadastradas no backoffice (<code>titulo</code> , <code>area</code> , <code>nivel</code> , <code>uf</code> , <code>cidade</code> , <code>status</code> , <code>slug</code> , etc.).
<code>documento_envios</code>	<code>DocumentoEnvio</code>	Cada disparo de Shortlist/Materiais (dados do cliente, mensagem, tipo de envio, identificador público).
<code>documento_currículos</code>	<code>DocumentoCurriculo</code>	Currículos/documentos anexados a um envio (<code>documento_envio_id</code>).
<code>users</code>	<code>User</code>	Usuários do backoffice (<code>status</code> , <code>super_admin</code>).
<code>password_resets</code> , <code>failed_jobs</code> , <code>personal_access_tokens</code> , <code>migrations</code>	—	Tabelas padrão do Laravel.

Todos os models de negócio (`Candidato`, `Candidatura`, `Vagas`) usam `protected $guarded = ['id']`, ou seja, **todos os campos são mass-assignable exceto id** — atenção redobrada ao aceitar dados de formulários/requests diretamente via `->fill($request->all())` ou `::create($request->all())`, pois isso é um ponto de atenção de segurança (mass assignment) a ser revisado em uma futura melhoria.

3.5 Armazenamento de arquivos (currículos e uploads)

Existem **dois mecanismos de upload diferentes** convivendo no projeto, importante entender para não confundi-los:

- Upload direto via `public_path()`** (usado em `SiteController@salvar_candidatura`): os currículos enviados pelo site público (candidatura a vaga e banco de currículos) são movidos diretamente para `public/uploads/curriculos/`, fora do sistema de disks do Laravel. O caminho relativo é salvo no campo `candidato.curriculo` (ex.: `uploads/curriculos/1743768235.pdf`).
- Upload via `Storage::disk('public')`** (usado em `HomeController@disparar_documentos / disparar_materiais`): planilhas e currículos anexados a

um envio de Shortlist/Materiais são gravados em `storage/app/public/documentos/...` e ficam acessíveis publicamente através do link simbólico `public/storage` → `storage/app/public`.

Esse link simbólico **precisa existir** para os downloads de Shortlist funcionarem. Ele é criado com:

```
php artisan storage:link
```

ou, alternativamente, acessando a rota utilitária `/fix-storage-link` já existente no projeto (útil em hosts compartilhados sem terminal).

3.6 Configurações sensíveis (`.env`)

O arquivo `.env` real do ambiente de produção/homologação **não deve ser versionado nem compartilhado publicamente** por conter credenciais. O projeto traz um `.env.example` como referência de estrutura. Ao restaurar o projeto em um novo ambiente, o novo desenvolvedor deve criar um `.env` próprio preenchendo, no mínimo:

- `APP_KEY` — gerado via `php artisan key:generate` (não deve ser copiado de outro ambiente).
- `APP_URL` — URL local, ex.: `http://localhost/dhcosta/public` ou domínio virtual configurado no WAMP.
- `DB_DATABASE` , `DB_USERNAME` , `DB_PASSWORD` — credenciais do banco local restaurado a partir do dump.
- `MAIL_MAILER` , `MAIL_HOST` , `MAIL_PORT` , `MAIL_USERNAME` , `MAIL_PASSWORD` , `MAIL_ENCRYPTION` , `MAIL_FROM_ADDRESS` , `MAIL_FROM_NAME` — necessários para o disparo de e-mails de Shortlist/Materiais funcionar de fato (sem isso, o envio falha silenciosamente ou lança exceção, dependendo da configuração de filas/log).
- `WHATSAPP_CONTACT` (usada em `config/services.php`) — número usado no botão de WhatsApp da home. Se não definida, assume o valor padrão hardcoded no `config/services.php` atual.

3.7 Passo a passo de restauração em ambiente local

O pacote entregue ao novo desenvolvedor contém **todas as pastas do projeto, incluindo `vendor/` e `node_modules/assets` já buildados quando aplicável**, além do dump SQL. A restauração é **manual** (sem `composer install` / `npm install` obrigatórios, embora recomendados caso o ambiente PHP/Node seja diferente do original). Passo a passo sugerido:

1. **Descompactar o projeto** dentro da pasta de sites do servidor local (ex.: `C:\wamp64\www\dhcosta` , ou pasta equivalente no XAMPP/Laragon).
2. **Criar o banco de dados** no MySQL local (ex.: `costad77_sistema`) usando um cliente como phpMyAdmin, HeidiSQL ou linha de comando.
3. **Importar o dump** `database/costad77_sistema.sql` para dentro do banco criado.
4. **Criar o arquivo `.env`** a partir do `.env.example` , ajustando os dados de conexão do banco (`DB_DATABASE` , `DB_USERNAME` , `DB_PASSWORD` , `DB_HOST` , `DB_PORT`) e do e-mail (ver seção 3.6).
5. **Gerar a chave da aplicação:** `php artisan key:generate`
6. **Conferir/installar dependências**, caso a pasta `vendor` enviada não seja compatível com a versão local de PHP: `composer install` (Normalmente **não será necessário**, pois o `vendor/` já vem no pacote

enviado.)

7. **Recriar o link simbólico de storage** (necessário para os downloads de Shortlist/Materiais funcionarem): `php artisan storage:link` ou acessar `http://localhost/dhcosta/public/fix-storage-link` no navegador.
8. **Conferir permissões de escrita** nas pastas `storage/`, `bootstrap/cache/`, `public/uploads/curriculos/` e `public/upload/curriculos/` (o servidor web precisa conseguir gravar arquivos nelas).
9. **Configurar o Virtual Host / apontar o document root** para a pasta `public/` do projeto (padrão Laravel). Em WAMP, isso normalmente é feito criando um VirtualHost apontando para `C:\wamp64\www\dhcosta\public`.
10. **Testar o login** no backoffice (`/login`) com um usuário existente no dump, e testar o fluxo público (home, candidatura a vaga, cadastro no banco de currículos) para validar que uploads e e-mails (se configurados) estão funcionando.
11. (Opcional) Rodar `php artisan config:clear` e `php artisan cache:clear` caso apareçam problemas de cache de configuração vindos do ambiente anterior.

Não é necessário rodar `php artisan migrate` neste processo — o dump SQL já contém toda a estrutura e os dados. Rodar as migrations padrão do Laravel sobre um banco já restaurado do dump pode gerar erro de "tabela já existe" (as tabelas `users`, `password_resets`, `failed_jobs`, `personal_access_tokens` já estão no dump).

4. Demanda pendente: repaginação do site

Existe uma demanda em aberto de **repaginação visual do site público** (troca de template HTML/CSS/JS), que será conduzida pelo novo desenvolvedor a partir de agora. O escopo funcional completo dessa demanda está detalhado no arquivo:

- [docs/escopo-repaginacao-site.doc](#) (também disponível em `.pdf`)

Resumo do que esse documento traz: - Mapeamento das 5 telas públicas do site (Home, Detalhe da Vaga/Candidatura, Banco de Talentos, Shortlist/Link Exclusivo, Política de Privacidade), com arquivo Blade e rota de cada uma. - Indicação de que a repaginação é **exclusivamente de camada visual** (HTML/CSS/JS) — nenhuma alteração em Controllers, Models, rotas ou banco de dados está prevista. - Lista de assets a **substituir** (`public/site/assets/css/`, `public/site/assets/js/`) e a **preservar** (`public/site/images/`, incluindo `new_brands/`). - Priorização da tela de Shortlist (Tela 4) por ser acessada por empresas clientes externas, com impacto direto na imagem da CostaDH.

O documento de **cronograma** dessa repaginação (com sprints, prazos e valores comerciais) **não será entregue** ao novo desenvolvedor por conter dados financeiros do orçamento acordado com a cliente. As informações estruturais relevantes desse cronograma (quais telas priorizar e em que ordem) já foram incorporadas ao resumo acima e à tabela de seções da home na seção 2.1-a deste documento.

5. Observações finais e recomendações

- **Conteúdo institucional só muda via código:** reforçando o que já foi dito na seção 2.1-a — a home não tem CMS. Qualquer pedido de alteração de texto/imagem da cliente deve ser feito editando `resources/views/site/index.blade.php` e os arquivos de imagem em `public/site/images/`.
- **Duas pastas de upload de currículo coexistem** (`public/uploads/curriculos/` e `public/upload/curriculos/`) — vale investigar com calma se ambas estão realmente em uso ou se uma é resquício de uma versão anterior, antes de fazer qualquer limpeza.
- **Mass assignment aberto** (`$guarded = [ 'id' ]`) nos models de negócio é um ponto de atenção de segurança para uma eventual revisão futura, especialmente nos formulários administrativos que usam `$request->all()` diretamente.
- **Sem tabela/enum de status de candidatura:** caso a evolução do painel de candidatos precise de regras mais rígidas (ex.: fluxo obrigatório de etapas), será necessário modelar isso, já que hoje o campo é texto livre.
- **Ambiente de e-mail precisa ser configurado** em qualquer ambiente novo para que o fluxo de Shortlist/Materiais funcione de ponta a ponta (ver seção 3.6).

Documento gerado em 04/09/2026 para fins de passagem de conhecimento técnico e funcional do projeto CostaDH.